
<Project Title>

Prepare by:

- 1- Name (Number)
- 2- Name (Number)

Supervisor

Dr.XXXX
First Semester 20232024

ABSTRACT

Table of Contents

1.Introduction	7
1.1.Project Description	7
1.2.Project Overview	7
1.3.Tasks	8
1.4.Project Planning	8
1.5.Planning of the Development Phases	9
1.6.The Scope of the Work	9
1.7.Stakeholders	10
1.8.Scope	10
1.9.Definitions, Acronyms, and Abbreviations	10
1.10.References	11
1.11.Overview	11
1.12.Product Scenarios	11
2.Requirements	12
2.1.Product Use Cases	12
2.2.Functional Requirements	12
2.2.1.User Authentication	12
2.3.Data Requirements	12
2.3.1.Submit Project Proposal	13
3.Design	14
3.1.System Design	14
3.2.Proposed Software Architecture	15
3.3.User Interface	17
3.4.Object Design	17
4.Test Plans	18
4.1.Approach	18
4.2.Test Cases	18
Scope	19
4.3.Definitions, Acronyms, and Abbreviations	19
4.4.References	19
4.5.Overview	19
5.Positioning	20
5.1.Business Opportunity	20
5.2.Problem Statement	20
5.3.Product Position Statement	20
6.Stakeholder and User Descriptions	20
6.1.Market Demographics	20
6.2.Stakeholder Summary	20
6.3.User Summary	21
6.4.User Environment	21
6.5.Stakeholder Profiles	21
6.5.1.<Stakeholder Name>	22
6.6.User Profiles	22
6.6.1.<User Name>	22
6.7.Key Stakeholder or User Needs	23

6.8.Alternatives and Competition	23
6.8.1.<aCompetitor>	23
6.8.2.<anotherCompetitor>	23
7.Product Overview	23
7.1.Product Perspective	23
7.2.Summary of Capabilities	23
7.3.Assumptions and Dependencies	24
7.4.Cost and Pricing	24
7.5.Licensing and Installation	24
8.Product Features	24
8.1.<aFeature>	25
8.2.<anotherFeature>	25
9.Constraints	25
10.Quality Ranges	25
11.Precedence and Priority	25
12.Other Product Requirements	25
12.1.Applicable Standards	25
12.2.System Requirements	25
12.3.Performance Requirements	25
12.4.Environmental Requirements	25
13.Documentation Requirements	25
13.1.User Manual	25
13.2.Online Help	26
13.3.Installation Guides, Configuration, and Read Me File	26
13.4.Labeling and Packaging	26
A Feature Attributes	26
A.1 Status	26
A.2 Benefit	26
A.3 Effort	27
A.4 Risk	27
A.5 Stability	27
A.6 Target Release	27
A.7 Assigned To	27
A.8 Reason	27

Table of Figure

Figure 1 Use Case Diagram	8
Figure 2 Schema	10
Figure 3 component diagram for the Submit Project Proposal	11
Figure 4 class diagram	12
Figure 5 Sequence Diagram	13

Chapter 1

1.Introduction

1.1.Project Description

Vibook is a full-stack lifestyle event booking platform developed as a graduation project in software engineering. The system connects consumers in Jordan with bookable business events while enabling approved business partners to publish and manage listings and bookings. A separate administrative web console supports platform operators in moderating businesses, events, bookings, ratings, and user reports.

The project addresses the need for a unified, trustworthy marketplace for timed experiences: users can discover events by governorate and category, view details, favorite listings, submit bookings, rate attended events, and report inappropriate content. Business owners apply for a business profile, await administrative approval, then create and manage events and incoming reservations. Administrators review applications, moderate catalog content, resolve reports, and view operational analytics.

Primary goals include delivering a secure JWT-authenticated REST API backed by MySQL, a cross-platform mobile client for guests and registered users, and a role-protected admin SPA. Success is measured by end-to-end implementation of catalog discovery, booking lifecycle, partner onboarding, moderation workflows, and stable integration between the three deployable components documented in the project repository.

1.2.Project Overview

The Vibook solution comprises three implemented parts: a Spring Boot backend API (Java 21, port 8080, prefix /api/v1), an Expo/React Native mobile application (file-based routing via expo-router), and a Vite/React admin dashboard for users with the ADMIN role. Persistent data is stored in MySQL database vibook_db using Hibernate JPA with sixteen entity types including User, BusinessProfile, BusinessEvent, Booking, Favorite, EventRating, ModerationReport, and reference data for governorates and categories.

[PLACEHOLDER: Figure 1.1 - Vibook System Architecture Diagram]

Description: Diagram the three clients (mobile, admin-web), Spring Boot API, JWT security filter, and MySQL vibook_db with major REST route groups (/auth, /events, /bookings, /business-profile, /business/events, /admin) as implemented in the repository.

Target users are guests (browse without login), registered consumers (USER role), business partners (USER with approved BusinessProfile and optional

ROLE_BUSINESS), and platform administrators. Core implemented capabilities include public and authenticated event catalog access, consumer bookings with PayPal Sandbox payment integration, favorites, event ratings, user moderation reports, business profile application and approval, partner event CRUD with photo upload, partner booking status updates, and admin analytics summary. Presentation-only mobile screens (premium membership, wallet, vouchers, legacy restaurant/stay routes) are outside the production data scope and are not described as backend-backed features in this document.

1.3.Tasks

The project was decomposed into verifiable engineering tasks aligned with the repository structure. Analysis and modeling tasks covered stakeholder needs, governorate/category taxonomy, and booking state transitions. Backend tasks included security configuration, JWT access and refresh tokens, twenty-three REST controllers, service-layer business rules, and JPA repositories with admin list specifications. Mobile tasks covered authentication hydration, Explore and search flows, event product-detail pages, checkout and payment, favorites, ratings, reports, Arabic/English localization with RTL layout, and the business partner hub under app/business/. Admin-web tasks included protected routing, Axios API client, and pages for businesses, events, bookings, ratings, reports, and dashboard charts. Table 1 summarizes the work breakdown schedule used for planning and tracking progress.

Table 1 Project Timeline

WBS	Task description	Start date	Finish date	Progress
1	Requirements analysis and domain modeling for event booking in Jordan	Sep 2024	Oct 2024	100%
2	System architecture and database design (MySQL, REST API)	Oct 2024	Nov 2024	100%
3	Backend implementation (Spring Boot, JWT, JPA entities)	Nov 2024	Feb 2025	100%
4	Mobile app implementation (Expo, consumer and partner flows)	Jan 2025	Apr 2025	100%
5	Admin web console and integration testing	Mar 2025	May 2025	100%

Table 1 Project Timeline

Table 1 lists the work breakdown structure, schedule, and completion status for major Vibook delivery tasks from requirements through documentation.

1.4.Project Planning

Project planning followed an incremental delivery model: establish the API contract and database schema first, then parallelize mobile and admin clients against stable endpoints. Risk areas identified early included JWT session refresh on mobile,

CORS configuration for LAN device testing, multipart upload limits for business event photos, and role separation between consumer, partner, and admin paths. Configuration is centralized in application.yml (datasource, JWT secret, upload directories, CORS patterns), mobile EXPO_PUBLIC_API_URL, and admin VITE_API_BASE_URL.

[PLACEHOLDER: Figure 1.2 - Project Gantt Chart]

Description: Gantt chart derived from Table 1 timeline showing analysis, backend, mobile, admin, and documentation phases with dependencies (API before client feature completion).

Quality activities included Spring Boot unit and integration tests, TypeScript type-checking on mobile and admin, and manual end-to-end verification of booking, approval, and moderation flows described in docs/GRADUATION_PROJECT_HANDOFF.md and docs/MOBILE_AND_ADMIN_GUIDE.md.

1.5.Planning of the Development Phases

Development phases were ordered to reduce integration risk:

Phase 1 — Foundation: MySQL schema via JPA entities, auth register/login/refresh/logout, governorate and category seed data, and public catalog reads.

Phase 2 — Consumer experience: mobile Explore, search, filters, event PDP, favorites, bookings API integration, ratings, and user reports.

Phase 3 — Business partner: business profile application and submit, admin approval, ROLE_BUSINESS lifecycle, partner event CRUD, time slots, photo uploads, and booking status updates.

Phase 4 — Administration: admin-web authentication, moderation queues, user management patches, and GET /admin/analytics/summary dashboard metrics.

Phase 5 — Stabilization: removal of runtime mock catalog in favor of API-only data, PayPal Sandbox checkout wiring, production data cleanup, and graduation documentation.

1.6.The Scope of the Work

The scope of work encompasses requirements engineering, design, implementation, and validation of the Vibook platform as a cohesive software product. Requirements were derived from the implemented codebase and verified against REST controllers, entity relationships, and mobile/admin route maps. Stakeholder and user needs—discovery, booking, partner self-service, and platform oversight—are addressed through concrete API and UI flows rather than hypothetical extensions.

This section establishes why the documented requirements exist: Jordan-focused event discovery, governed business onboarding, and trust mechanisms (ratings and moderation). Detailed functional and non-functional requirements appear in subsequent chapters; stakeholder profiles and user environment descriptions are expanded in Chapter 3 of this template.

1.7.Stakeholders

Key stakeholders for Vibook include:

- *End consumers and guests — browse and book events; require accurate catalog data, secure accounts, and reliable booking confirmation.*
- *Business partners — submit BusinessProfile applications, manage events and bookings after approval; require transparent status and listing tools.*
- *Platform administrators — operate admin-web; approve businesses, moderate events and reports, and monitor analytics.*
- *Academic supervisor and evaluation committee — assess software engineering process, architecture, and documentation completeness.*
- *Development team — implement and maintain backend, mobile, and admin-web codebases under version control.*

[PLACEHOLDER: Table 1.1 - Stakeholder Summary]

Description: Tabular summary of each stakeholder type with representative role, responsibilities, success criteria, and involvement level in requirements and acceptance testing.

1.8.Scope

In scope for this graduation project:

- *Spring Boot 3.3 REST API with Spring Security, JWT, JPA, and MySQL persistence.*
- *Mobile app: authentication, Explore/search/filters, event PDP, bookings (including PayPal Sandbox), favorites, ratings, reports, business partner hub, bilingual UI (English/Arabic) with RTL support.*
- *Admin-web: JWT login for ADMIN role, business approval, event and booking moderation, ratings and reports queues, analytics summary.*
- *Static file serving for profile images and business logos/banners.*

Explicitly out of scope (not implemented as production backend features):

- *Premium membership billing, wallet balances, and voucher redemption beyond static mobile UI.*
- *Legacy mobile PDP routes for restaurant, stay, experience, package, and organizer shells.*
- *Push notification delivery infrastructure (notification list is presentation-only).*

Future work may extend payment providers beyond Sandbox, enrich notifications, and expand catalog types; such items are not claimed as delivered functionality in this report.

1.9.Definitions, Acronyms, and Abbreviations

API — Application Programming Interface; Vibook exposes REST endpoints under /api/v1.

PDP — Product Detail Page; mobile route app/event/[id].tsx for a single event listing.

JWT — JSON Web Token; access tokens (24h) and refresh tokens (30 days, stored server-side).

JPA — Java Persistence API; Hibernate maps entities to MySQL tables.

SPA — Single Page Application; admin-web built with Vite and React Router.

Expo — Framework and toolchain for the React Native mobile application.

DTO — Data Transfer Object; request/response records in backend/dto.

CORS — Cross-Origin Resource Sharing; configured for localhost and LAN origins.
RTL — Right-to-left layout for Arabic locale via `I18nManager` and `RtlLayout`.
JOD — Jordanian Dinar; default currency display context in mobile locale settings.
USER / ADMIN / ROLE_BUSINESS — Spring Security roles used in authorization rules.
BusinessProfile — Partner application entity with approval workflow before event publishing.
ModerationReport — User-submitted report queued for admin resolution.

1.10. References

- [1] Spring Boot Documentation, version 3.3 — <https://docs.spring.io/spring-boot/docs/3.3.x/reference/html/>
- [2] Spring Security Reference — <https://docs.spring.io/spring-security/reference/>
- [3] Spring Data JPA Documentation — <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>
- [4] Hibernate ORM User Guide — <https://docs.hibernate.org/orm/documentation/>
- [5] MySQL 8.0 Reference Manual — <https://dev.mysql.com/doc/refman/8.0/en/>
- [6] JWT (Java JWT) library, version 0.12.6 — <https://github.com/jwt/jwt>
- [7] Expo Documentation, SDK 54 — <https://docs.expo.dev/>
- [8] React Native Documentation, version 0.81 — <https://reactnative.dev/docs/getting-started>
- [9] expo-router Documentation — <https://docs.expo.dev/router/introduction/>
- [10] Zustand State Management — <https://github.com/pmndrs/zustand>
- [11] React 19 Documentation — <https://react.dev/>
- [12] Vite Documentation, version 8 — <https://vite.dev/guide/>
- [13] Axios HTTP Client — <https://axios-http.com/docs/intro>
- [14] React Router Documentation, version 7 — <https://reactrouter.com/>
- [15] Recharts Documentation — <https://recharts.org/en-US/>
- [16] PayPal REST API (Sandbox) — <https://developer.paypal.com/api/rest/>
- [17] Jakarta Bean Validation — <https://jakarta.ee/specifications/bean-validation/>
- [18] TypeScript Handbook — <https://www.typescriptlang.org/docs/>
- [19] RFC 7519 — JSON Web Token (JWT) — <https://datatracker.ietf.org/doc/html/rfc7519>
- [20] Vibook project repository — docs/GRADUATION_PROJECT_HANDOFF.md, docs/MOBILE_AND_ADMIN_GUIDE.md

1.11. Overview

This chapter introduces the Vibook graduation project: its purpose as a Jordan-focused event booking marketplace, the three-part architecture (mobile, backend API, admin console), and the planning structure used to deliver verified functionality. Subsequent chapters of this document detail positioning, stakeholder and user descriptions, product capabilities, constraints, and technical design artifacts including use cases, database schema, and diagrams.

Readers should use Section 1.8 Scope together with the implementation handoff documentation to distinguish production API-backed features from presentation-only mobile screens. Figures and tables marked as placeholders in this chapter should be replaced with captures and diagrams from the running system and database before final submission.

1.12. Product Scenarios

The following product scenario summarizes primary interactions between actors and the implemented Vibook system. Consumers and guests interact with the mobile app; business partners use mobile business routes after profile approval; administrators use admin-web; all privileged operations are enforced by the backend JWT authorization model.

[PLACEHOLDER: Figure 1.3 - Vibook Use Case Diagram]

Description: Show actors Guest, Consumer (USER), Business Partner, and Admin interacting with use cases Discover Events, Book Event, Manage Business Profile, Publish Event, Moderate Content, and View Analytics based on implemented mobile and admin flows.

Figure 1.3 Vibook Use Case Diagram

Figure 1.3 illustrates primary interactions between Vibook actors and the implemented booking, partner onboarding, and administration capabilities. Replace the placeholder with the project-specific diagram before submission.

Chapter 2

2.Requirements

2.1.Product Use Cases

List and describe individual use cases, including actors, preconditions, postconditions, and main flow.

Table 2 Submit Project Proposal Use Case

Use Case	<i>Submit Project Proposal</i>
Actor	<i>Student</i>
Preconditions	<i>The student must be logged into the Graduation Project system.</i>
Postconditions	<i>The project proposal is submitted for review</i>
Main Flow	<i>1. The student logs into the system 2. The student navigates to the project proposal submission page. 3. The student fills out the required information and submits the proposal.</i>

2.2.Functional Requirements

Specify the functional capabilities and features of the software. Describe each requirement in detail, including its purpose and acceptance criteria.

2.2.1.User Authentication

Description: Users should be able to create accounts and log in securely.

Purpose: This feature ensures that only authorized users can access the system, maintaining data security and privacy.

- Acceptance Criteria:
 - 1) Users can register for an account by providing necessary information such as username, email, and password.
 - 2) Registered users can log in using their credentials.
 - 3) Passwords are securely hashed and stored in the database.

2.3.Data Requirements

Identify the types of data the system will handle. Define data entities, attributes, relationships, and constraints.

Specify data storage and retrieval mechanisms.

2.3.1. Submit Project Proposal

Types of Data:

- I. Student data: Information about the student submitting the proposal.
- II. Project proposal data: Details about the proposed project, including title, description, objectives, methodology, resources required, and timeline.
- III. Supporting documents: Any additional files or documents attached to the proposal.

Data Entities:

- I. Student: Represents a student user of the system.
- II. Project Proposal: Represents the proposal submitted by the student.
- III. Supporting Document: Represents any additional files or documents attached to the proposal.

Attributes:

- I. Student:
 - 1) Name
 - 2) Student ID
 - 3) Email
 - 4) Department
- II. Project Proposal:
 - 1) Title
 - 2) Description
 - 3) Objectives
 - 4) Methodology
 - 5) Resources required
 - 6) Timeline
 - 7) Status
 - 8) Submission date
- III. Supporting Document:
 - 1) File name
 - 2) File type
 - 3) File size
 - 4) Upload date

Constraints

- 1) Unique Student ID: Each student should have a unique identifier.
- 2) Required Proposal Fields: Certain fields in the project proposal form may be mandatory, such as title, description, and objectives.
- 3) File Size Limit: There may be a constraint on the maximum size of supporting documents that can be uploaded.
- 4) Submission Deadline: A deadline constraint may be imposed for submitting project proposals.



Figure 2 Database Schema

Chapter 3

3.Design

3.1.System Design

Define the high-level design goals and principles. Create a component diagram showing the major components of the system and their interactions.

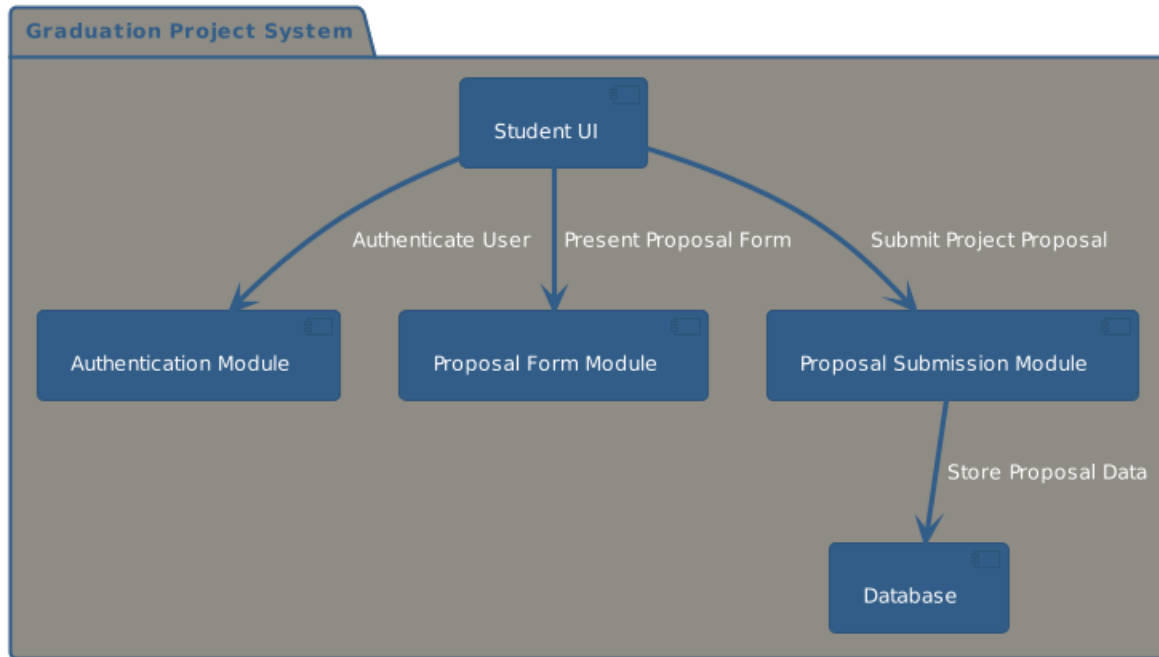


Figure 3 component diagram for the Submit Project Proposal

3.2. Proposed Software Architecture

Provide an overview of the software architecture, including layers, components, and their interactions. Create class diagrams showing the static structure of the system. Develop sequence diagrams illustrating dynamic interactions between objects or components.

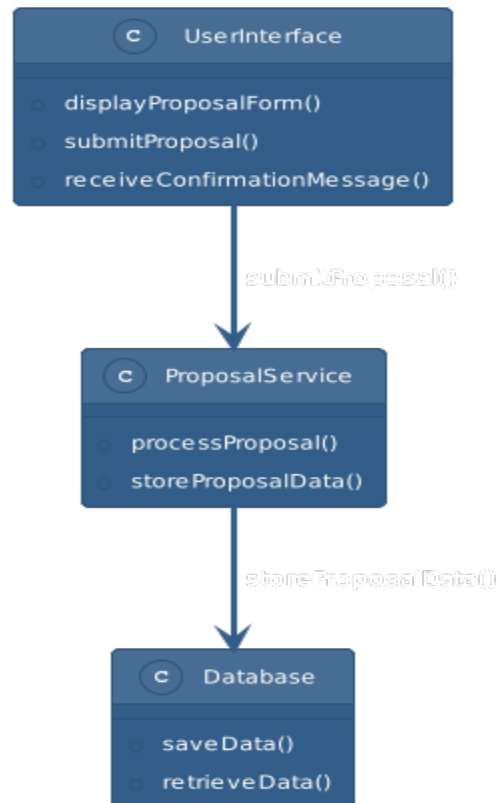


Figure 4 class diagram.

*In this class diagram, **UserInterface** represents the user interface component with methods for displaying the proposal form, submitting proposals, and receiving confirmation messages. **ProposalService** represents the service component responsible for processing proposals and storing proposal data. **Database** represents the data access component with methods for saving and retrieving data.*

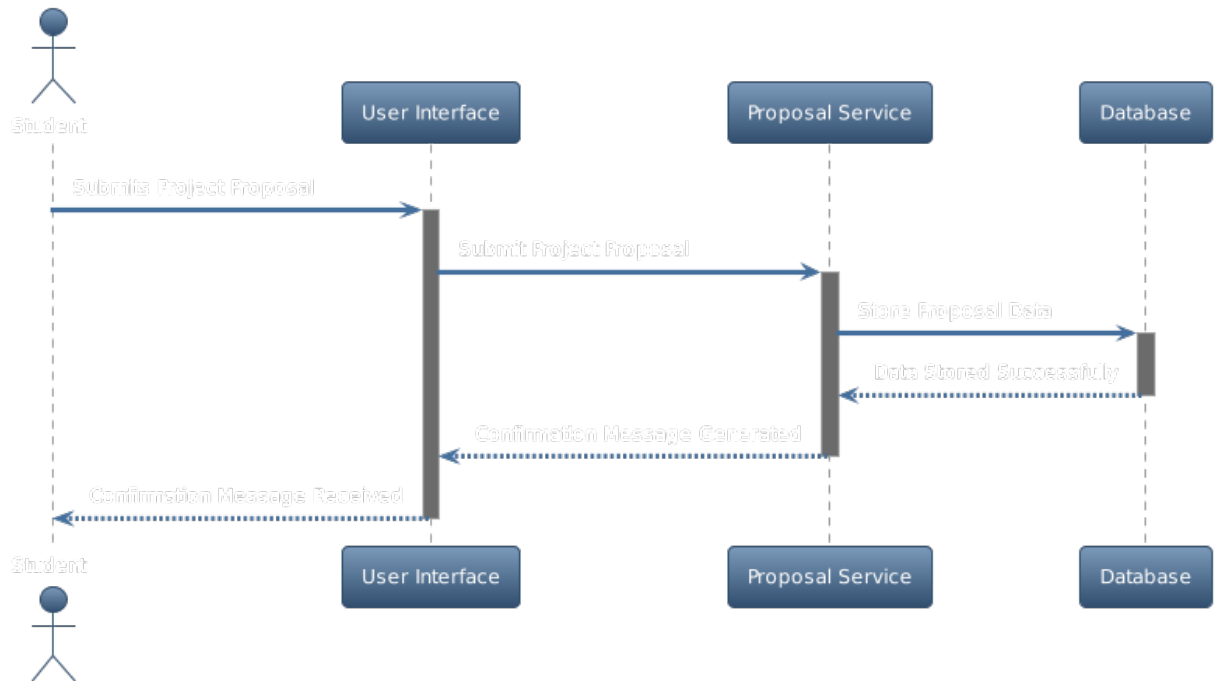


Figure 5 Sequence Diagram

The figure above presents the sequence diagram for

UserInterface initiates the proposal submission process by calling the submitProposal() method. ProposalService receives the submission request and calls the storeProposalData() method to store the proposal data in the database. The Database component stores the data and sends a confirmation message back to the ProposalService. Finally, the ProposalService sends the confirmation message to the UserInterface.

3.3.User Interface

Design the graphical user interface (GUI) to be intuitive and user-friendly. Create wireframe diagrams or mockups to visualize the layout and design of the interface.

3.4.Object Design

Define the objects and their attributes and methods. Specify how objects interact with each other and with the system.

Chapter 4

4. Test Plans

4.1. Approach

Describe the overall approach to testing, including the types of tests to be conducted.

Specify testing materials and resources required.

Define criteria for suspending and resuming testing activities.

4.2. Test Cases

Create individual test cases for each functional requirement or use case.

Specify input data, expected outcomes, and pass/fail criteria for each test case.

Organize test cases into a test suite for efficient execution and management.

This comprehensive outline should guide you through the process of creating a detailed project plan for your Bachelor's software engineering project, ensuring that all relevant aspects are covered effectively.

Scope

*[A brief description of the scope of this **Vision** document; what Project(s) it is associated with and anything else that is affected or influenced by this document.]*

4.3. Definitions, Acronyms, and Abbreviations

*[This subsection provides the definitions of all terms, acronyms, and abbreviations required to properly interpret the **Vision** document. This information may be provided by reference to the project's Glossary.]*

4.4. References

*[This subsection provides a complete list of all documents referenced elsewhere in the **Vision** document. Identify each document by title, report number if applicable, date, and publishing organization. Specify the sources from which the references can be obtained. This information may be provided by reference to an appendix or to another document.]*

4.5. Overview

*[This subsection describes what the rest of the **Vision** document contains and explains how the document is*

organized.]

5.Positioning

5.1.Business Opportunity

[Briefly describe the business opportunity being met by this project.]

5.2.Problem Statement

[Provide a statement summarizing the problem being solved by this project. The following format may be used:]

The problem of	<i>[describe the problem]</i>
affects	<i>[the stakeholders affected by the problem]</i>
the impact of which is	<i>[what is the impact of the problem?]</i>
a successful solution would be	<i>[list some key benefits of a successful solution]</i>

5.3.Product Position Statement

[Provide an overall statement summarizing, at the highest level, the unique position the product intends to fill in the marketplace. The following format may be used:]

For	<i>[target customer]</i>
Who	<i>[statement of the need or opportunity]</i>
The (product name)	<i>is a [product category]</i>
That	<i>[statement of key benefit; that is, the compelling reason to buy]</i>
Unlike	<i>[primary competitive alternative]</i>
Our product	<i>[statement of primary differentiation]</i>

[A product position statement communicates the intent of the application and the importance of the project to all concerned personnel.]

6.Stakeholder and User Descriptions

[To effectively provide products and services that meet your stakeholders' and users' real needs, it is necessary to identify and involve all of the stakeholders as part of the Requirements Modeling process. You must also identify the users of the system and ensure that the stakeholder community adequately represents them. This section provides a profile of the stakeholders and users involved in the project, and the key problems that they perceive to be addressed by the proposed solution. It does not describe their specific requests or requirements as these are captured in a separate stakeholder requests artifact. Instead, it provides the background and justification for why the requirements are needed.]

6.1.Market Demographics

[Summarize the key market demographics that motivate your product decisions. Describe and position target market segments. Estimate the market's size and growth by using the number of potential users or the amount of money your customers spend trying to meet needs that your product or enhancement would fulfill. Review major industry trends and technologies. Answer these strategic questions:]

- *What is your organization's reputation in these markets?*
- *What would you like it to be?*
- *How does this product or service support your goals?]*

6.2.Stakeholder Summary

[There are a number of stakeholders with an interest in the development and not all of them are end users. Present a

summary list of these non-user stakeholders. (The users are summarized in section 3.3.)]

Name	Description	Responsibilities
<i>[Name the stakeholder type.]</i>	<i>[Briefly describe the stakeholder.]</i>	<i>[Summarize the stakeholder's key responsibilities with regard to the system being developed; that is, their interest as a stakeholder. For example, this stakeholder:</i> <i>- ensures that the system will be maintainable</i> <i>- ensures that there will be a market demand for the product's features</i> <i>- monitors the project's progress</i> <i>- approves funding</i> <i>- and so forth]</i>

6.3. User Summary

[Present a summary list of all identified users.]

Name	Description	Responsibilities	Stakeholder
<i>[Name the user type.]</i>	<i>[Briefly describe what they represent with respect to the system.]</i>	<i>[List the user's key responsibilities with regard to the system being developed; for example:</i> <i>- captures details</i> <i>- produces reports</i> <i>- coordinates work</i> <i>- and so on]</i>	<i>[If the user is not directly represented, identify which stakeholder is responsible for representing the user's interest.]</i>

6.4. User Environment

[Detail the working environment of the target user. Here are some suggestions:

- Number of people involved in completing the task? Is this changing?*
- How long is a task cycle? Amount of time spent in each activity? Is this changing?*
- Any unique environmental constraints: mobile, outdoors, in-flight, and so on?*
- Which systems platforms are in use today? Future platforms?*
- What other applications are in use? Does your application need to integrate with them?*

This is where extracts from the Business Model could be included to outline the task and roles involved and so on.]

6.5. Stakeholder Profiles

[Describe each stakeholder in the system here by filling in the following table for each stakeholder. Remember that stakeholder types can be as divergent as users, departments, and technical developers. A thorough profile would cover the following topics for each type of stakeholder:]

6.5.1.<Stakeholder Name>

Representative	<i>[Who is the stakeholder representative to the project? (Optional if documented elsewhere.) What we want here is names.]</i>
Description	<i>[A brief description of the stakeholder type.]</i>
Type	<i>[Qualify the stakeholder's expertise, technical background, and degree of sophistication—that is, guru, business, expert, casual user, and so on.]</i>
Responsibilities	<i>[List the stakeholder's key responsibilities with regard to the system being developed—that is, their interest as a stakeholder.]</i>
Success Criteria	<i>[How does the stakeholder define success? How is the stakeholder rewarded?]</i>
Involvement	<i>[How is the stakeholder involved in the project? Relate where possible to Rational Unified Process roles—that is, Requirements Reviewer and so on.]</i>
Deliverables	<i>[Are there any additional deliverables required by the stakeholder? These could be project deliverables or outputs from the system under development.]</i>
Comments / Issues	<i>[Problems that interfere with success and any other relevant information go here.]</i>

6.6.User Profiles

[Describe each unique user of the system here by filling in the following table for each user type. Remember user types can be as divergent as gurus and novices. For example, a guru might need a sophisticated, flexible tool with cross-platform support, while a novice might need a tool that is easy to use and user-friendly. A thorough profile needs to cover the following topics for each type of user.]

6.6.1.<User Name>

Representative	<i>[Who is the user representative to the project? (Optional if documented elsewhere.) This often refers to the Stakeholder that represents the set of users, for example, Stakeholder1.]</i>
Description	<i>[A brief description of the user type.]</i>
Type	<i>[Qualify the user's expertise, technical background, and degree of sophistication—that is, guru, casual user, and so on.]</i>
Responsibilities	<i>[List the user's key responsibilities with regard to the system being developed—that is, captures details, produces reports, coordinates work, and so forth.]</i>
Success Criteria	<i>[How does the user define success? How is the user rewarded?]</i>
Involvement	<i>[How is the user involved in the project? Relate where possible to Rational Unified Process roles—that is, Requirements Reviewer, and so on.]</i>
Deliverables	<i>[Are there any deliverables the user produces and, if so, for whom?]</i>
Comments / Issues	<i>[Problems that interfere with success and any other relevant information go here. These would include trends that make the user's job easier or harder.]</i>

6.7.Key Stakeholder or User Needs

[List the key problems with existing solutions as perceived by the stakeholder or user. Clarify the following issues for each problem:]

- *What are the reasons for this problem?*
- *How is it solved now?*
- *What solutions does the stakeholder or user want?]*

*[It is important to understand the **relative** importance the stakeholder or user places on solving each problem. Ranking and cumulative voting techniques indicate problems that **must** be solved versus issues they would like addressed.*

Fill in the following table—if using Rational RequisitePro to capture the Needs, this could be an extract or report from that tool.]

Need	Priority	Concerns	Current Solution	Proposed Solutions
Broadcast messages				

6.8.Alternatives and Competition

[Identify alternatives the stakeholder perceives as available. These can include buying a competitor's product, building a homegrown solution or simply maintaining the status quo. List any known competitive choices that exist or may become available. Include the major strengths and weaknesses of each competitor as perceived by the stakeholder or end user.]

6.8.1.<aCompetitor>

6.8.2.<anotherCompetitor>

7.Product Overview

[This section provides a high level view of the product capabilities, interfaces to other applications, and system configurations. This section usually consists of three subsections, as follows:]

- *Product perspective*
- *Product functions*
- *Assumptions and dependencies]*

7.1.Product Perspective

*[This subsection of the **Vision** document puts the product in perspective to other related products and the user's environment. If the product is independent and totally self-contained, state it here. If the product is a component of a larger system, then this subsection needs to relate how these systems interact and needs to identify the relevant interfaces between the systems. One easy way to display the major components of the larger system, interconnections, and external interfaces is with a block diagram.]*

7.2.Summary of Capabilities

*[Summarize the major benefits and features the product will provide. For example, a **Vision** document for a customer support system may use this part to address problem documentation, routing, and status reporting without mentioning the amount of detail each of these functions requires.*

Organize the functions so the list is understandable to the customer or to anyone else reading the document for the first time. A simple table listing the key benefits and their supporting features might suffice. For example:]

Table 4-1 Customer Support System

Customer Benefit	Supporting Features
New support staff can quickly get up to speed.	Knowledge base assists support personnel in quickly identifying known fixes and workarounds.
Customer satisfaction is improved because nothing falls through the cracks.	Problems are uniquely itemized, classified and tracked throughout the resolution process. Automatic notification occurs for any aging issues.
Management can identify problem areas and gauge staff workload.	Trend and distribution reports allow high level review of problem status.
Distributed support teams can work together to solve problems.	Replication server allows current database information to be shared across the enterprise.
Customers can help themselves, lowering support costs and improving response time.	Knowledge base can be made available over the Internet. Includes hypertext search capabilities and graphical query engine.

7.3.Assumptions and Dependencies

*[List each of the factors that affect the features stated in the **Vision** document. List assumptions that, if changed, will alter the **Vision** document. For example, an assumption may state that a specific operating system will be available for the hardware designated for the software product. If the operating system is not available, the **Vision** document will need to change.]*

7.4.Cost and Pricing

[For products sold to external customers and for many in-house applications, cost and pricing issues can directly impact the application's definition and implementation. In this section, record any cost and pricing constraints that are relevant. For example, distribution costs, (# of diskettes, # of CD-ROMs, CD mastering) or other cost of goods sold constraints (manuals, packaging) may be material to the projects success, or irrelevant, depending on the nature of the application.]

7.5.Licensing and Installation

[Licensing and installation issues can also directly impact the development effort. For example, the need to support serializing, password security or network licensing will create additional requirements of the system that must be considered in the development effort.

Installation requirements may also affect coding or create the need for separate installation software.]

8.Product Features

[List and briefly describe the product features. Features are the high-level capabilities of the system that are necessary to deliver benefits to the users. Each feature is an externally desired service that typically requires a series of inputs to achieve the desired result. For example, a feature of a problem tracking system might be the ability to provide trending reports. As the use-case model takes shape, update the description to refer to the use cases.

*Because the **Vision** document is reviewed by a wide variety of involved personnel, the level of detail needs to be general enough for everyone to understand. However, enough detail must be available to provide the team with the information they need to create a use-case model.*

To effectively manage application complexity, we recommend for any new system, or an increment to an existing system, capabilities are abstracted to a high enough level so 25-99 features result. These features provide the

fundamental basis for product definition, scope management, and project management. Each feature will be expanded in greater detail in the use-case model.

Throughout this section, each feature will be externally perceivable by users, operators or other external systems. These features need to include a description of functionality and any relevant usability issues that must be addressed. The following guidelines apply:

- Avoid design. Keep feature descriptions at a general level. Focus on capabilities needed and why (not how) they should be implemented.*
- If you are using the Rational RequisitePro toolkit, all need to be selected as requirements of type for easy reference and tracking.]*

8.1.<aFeature>

8.2.<anotherFeature>

9.Constraints

[Note any design constraints, external constraints or other dependencies.]

10.Quality Ranges

[Define the quality ranges for performance, robustness, fault tolerance, usability, and similar characteristics that are not captured in the Feature Set.]

11.Precedence and Priority

[Define the priority of the different system features.]

12.Other Product Requirements

[At a high level, list applicable standards, hardware or platform requirements, performance requirements, and environmental requirements.]

12.1.Applicable Standards

[List all standards with which the product must comply. These can include legal and regulatory (FDA, UCC) communications standards (TCP/IP, ISDN), platform compliance standards (Windows, UNIX, and so on), and quality and safety standards (UL, ISO, CMM).]

12.2.System Requirements

[Define any system requirements necessary to support the application. These can include the supported host operating systems and network platforms, configurations, memory, peripherals, and companion software.]

12.3.Performance Requirements

[Use this section to detail performance requirements. Performance issues can include such items as user load factors, bandwidth or communication capacity, throughput, accuracy, and reliability or response times under a variety of loading conditions.]

12.4.Environmental Requirements

[Detail environmental requirements as needed. For hardware- based systems, environmental issues can include temperature, shock, humidity, radiation, and so forth. For software applications, environmental factors can include usage conditions, user environment, resource availability, maintenance issues, and error handling and recovery.]

13.Documentation Requirements

[This section describes the documentation that must be developed to support successful application deployment.]

13.1.User Manual

[Describe the purpose and contents of the User Manual. Discuss desired length, level of detail, need for index,

glossary of terms, tutorial versus reference manual strategy, and so on. Formatting and printing constraints must also be identified.]

13.2.Online Help

[Many applications provide an online help system to assist the user. The nature of these systems is unique to application development as they combine aspects of programming (hyperlinks, and so forth) with aspects of technical writing, such as organization and presentation. Many have found the development of an online help system is a project within a project that benefits from up-front scope management and planning activity.]

13.3.Installation Guides, Configuration, and Read Me File

[A document that includes installation instructions and configuration guidelines is important to a full solution offering. Also, a Read Me file is typically included as a standard component. The Read Me file can include a "What's New With This Release" section, and a discussion of compatibility issues with earlier releases. Most users also appreciate documentation defining any known bugs and workarounds in the Read Me file.]

13.4.Labeling and Packaging

[Today's state-of-the-art applications provide a consistent look and feel that begins with product packaging and manifests through installation menus, splash screens, help systems, GUI dialogs, and so on. This section defines the needs and types of labeling to be incorporated into the code. Examples include copyright and patent notices, corporate logos, standardized icons and other graphic elements, and so forth.]

A Feature Attributes

[Features are given attributes that can be used to evaluate, track, prioritize, and manage the product items proposed for implementation. All requirement types and attributes need to be outlined in the Requirements Management Plan, however, you may wish to list and briefly describe the attributes for features that have been chosen. The following subsections represent a set of suggested feature attributes.]

A.1 Status

[Set after negotiation and review by the project management team. Tracks progress during definition of the project baseline.]

Proposed	<i>[Used to describe features that are under discussion but have not yet been reviewed and accepted by the "official channel," such as a working group consisting of representatives from the project team, product management, and user or customer community.]</i>
Approved	<i>[Capabilities that are deemed useful and feasible, and have been approved for implementation by the official channel.]</i>
Incorporated	<i>[Features incorporated into the product baseline at a specific point in time.]</i>

A.2 Benefit

[Set by Marketing, the product manager or the business analyst. All requirements are not created equal. Ranking requirements by their relative benefit to the end user opens a dialog with customers, analysts, and members of the development team. Used in managing scope and determining development priority.]

Critical	<i>[Essential features. Failure to implement means the system will not meet customer needs. All critical features must be implemented in the release or the schedule will slip.]</i>
Important	<i>[Features important to the effectiveness and efficiency of the system for most applications. The functionality cannot be easily provided in some other way. Lack of inclusion of an important feature may affect customer or user satisfaction, or even revenue, but release will not be delayed due to lack of any important feature.]</i>
Useful	<i>[Features that are useful in less typical applications will be used less frequently or for which reasonably efficient workarounds can be achieved. No significant revenue or customer satisfaction impact can be expected if such an item is not included in a release.]</i>

A.3 Effort

[Set by the development team. Because some features require more time and resources than others, estimating the number of team or person-weeks, lines of code required or function points, for example, is the best way to gauge complexity and set expectations of what can and cannot be accomplished in a given time frame. Used in managing scope and determining development priority.]

A.4 Risk

[Set by development team based on the probability the project will experience undesirable events, such as cost overruns, schedule delays or even cancellation. Most project managers find categorizing risks, as high, medium, and low, is sufficient, although finer gradations are possible. Risk can often be indirectly assessed by measuring the uncertainty (range) of the projects team's schedule estimate.]

A.5 Stability

[Set by the analyst and development team, this is based on

the probability that features will change or the team's understanding of the feature will change. Used to help establish development priorities and determine those items for which additional elicitation is the appropriate next action.]

A.6 Target Release

*[Records the intended product version in which the feature will first appear. This field can be used to allocate features from a **Vision** document into a particular baseline release. When combined with the status field, your team can propose, record, and discuss various features of the release without committing them to development. Only features whose Status is set to Incorporated and whose Target Release is defined will be implemented. When scope management occurs, the Target Release Version Number can be increased so the item will remain in the **Vision** document but will be scheduled for a later release.]*

A.7 Assigned To

[In many projects, features will be assigned to "feature teams" responsible for further elicitation, writing the software requirements, and implementation. This simple pull-down list will help everyone on the project team to understand responsibilities better.]

A.8 Reason

[This text field is used to track the source of the requested feature. Requirements exist for specific reasons. This field records an explanation or a reference to an explanation. For example, the reference might be to a page and line number of a product requirement specification or to a minute marker on a video of an important customer review.]